

Experiment No. _____

Online Cab Booking System

(SRS Report)

JAMIA MILLIA ISLAMIA
Department of Computer Engineering
Bachelor of Technology

Members:

- | | |
|-----------------|-------------|
| 1. Abdul Nasood | Roll No. 2 |
| 2. Md Nehal | Roll No. 38 |

Table of Contents

1. Introduction.....	1
1.1 Purpose.....	1
1.2 Product Scope.....	1
1.3 Technologies to be Used.....	1
1.4 Definitions, Acronyms and Abbreviations.....	1
2. Overall Description.....	1
2.1 Product Perspective.....	1
2.2 User Classes and Characteristics.....	1
2.3 Operating Environment.....	1
2.4 Software Interfaces.....	2
2.5 Hardware Requirements.....	2
2.6 Functionalities.....	2
2.7 General Constraints.....	2
2.8 Assumptions and Dependencies.....	2
3. System Features.....	2
3.1 Functional Requirements.....	2
3.2 Technical Issues.....	3
3.3 Non-Functional Requirements.....	3
3.4 Business Rules.....	3
3.5 Fare Calculation.....	3
4. External Interface Requirements.....	4
4.1 User Interfaces.....	4
4.2 Hardware Interfaces.....	4
4.3 Communication Interfaces.....	4
4.4 API Endpoints.....	4
5. Design.....	5
5.1 Use Case Diagram.....	5
5.2 Use Case Descriptions.....	6
5.3 Ride Status Flow.....	6
5.4 Activity Diagram: Booking a Ride.....	7
5.5 Activity Diagram: Handling a Ride.....	8
5.6 Data Flow Diagrams (DFD 0 and DFD 1).....	9
5.7 DFD Level 2: Ride Management.....	10
6. Database Design.....	11
6.1 Tables.....	11
6.2 Relationships.....	11
7. Testing.....	11
7.1 Test Cases.....	11
7.2 Requirement Traceability.....	12
8. Risk Analysis.....	12
9. Future Scope.....	12
10. Conclusion.....	12
11. References.....	12

1. Introduction

1.1 Purpose

This SRS describes what our Online Cab Booking System should do. We wrote it so that everyone working on the project, and anyone reviewing it, has one clear reference for its features, rules and design.

1.2 Product Scope

The app lets people book a cab without calling anyone or waiting on the road. A user enters the pickup and drop points, picks a vehicle, watches the cab arrive on the map and pays when the ride ends. Drivers get ride requests on their phone, and an admin looks after accounts, payments and bookings.

1.3 Technologies to be Used

Part	What We Use
Front-end	React.js with HTML, CSS and JavaScript
Back-end	Node.js with Express
Database	PostgreSQL
Live Updates	Socket.IO for live location and ride status
Maps	Google Maps API
Payments	Razorpay (UPI, cards, wallets) and cash
Notifications	SMS, email and push alerts

1.4 Definitions, Acronyms and Abbreviations

Term	Meaning
SRS	Software Requirements Specification
FR / NFR	Functional / Non-Functional Requirement
API	Application Programming Interface
JWT	JSON Web Token, used to keep users logged in
DFD	Data Flow Diagram
GPS	Global Positioning System
ETA	Estimated Time of Arrival
OTP	One-Time Password

2. Overall Description

2.1 Product Perspective

It's a standalone web app. Things that usually need a phone call or a trip to a taxi stand, like finding a cab, agreeing on the fare and paying, all happen in one place.

2.2 User Classes and Characteristics

- **User:** books, tracks and pays for rides. Basic phone skills are enough.
- **Driver:** accepts rides, follows the directions and collects cash. They use the app while driving, so the screens have to be simple.
- **Administrator:** manages accounts, payments and bookings from an admin panel.

2.3 Operating Environment

Any modern browser (Chrome, Firefox, Safari or Edge) on a phone or computer. The server runs on a Linux cloud machine, for example on AWS.

2.4 Software Interfaces

PostgreSQL stores the data, the React front-end talks to the server through a REST API, Google Maps handles locations and Razorpay handles online payments.

2.5 Hardware Requirements

- **Server:** 4-core CPU, 16 GB RAM and 500 GB SSD.
- **Client:** any phone or computer with internet. Drivers need GPS turned on.

2.6 Functionalities

The main features are signing up with an OTP, booking a ride with a fare estimate, sending the request to the nearest free driver, live tracking, paying online or in cash, cancelling rides and rating drivers.

2.7 General Constraints

- Personal, location and payment data must stay private.
- The app has to cope with rush-hour traffic.
- Only drivers with a valid licence and vehicle papers can take rides.
- Online payments must go through an RBI-approved gateway.

2.8 Assumptions and Dependencies

- Users and drivers have smartphones with internet and GPS.
- Google Maps, Razorpay and the SMS service are up and running.
- Drivers keep their location on while they are online.
- The admin checks every driver's documents before approving them.

3. System Features

3.1 Functional Requirements

ID	Area	Requirement
FR-01	Account	Users sign up with their name, phone and password, verified by OTP.
FR-02	Account	Drivers also upload licence, RC and insurance and need admin approval.
FR-03	Account	Everyone can log in, log out and reset a forgotten password.
FR-04	Booking	The user sets pickup and drop by typing an address or dropping a pin on the map.
FR-05	Booking	Before booking, the app shows the fare and ETA for each vehicle type.
FR-06	Booking	After booking, the user sees the driver's name, photo, car number and rating.
FR-07	Booking	The user can cancel any time before the ride starts.
FR-08	Ride	The request goes to the nearest online driver with the right vehicle type.
FR-09	Ride	The driver accepts or rejects. No reply in time means the next driver gets it.
FR-10	Ride	The ride starts only after the driver enters the OTP shown on the user's phone.
FR-11	Tracking	The user sees the cab live on the map, and the driver gets turn-by-turn directions.
FR-12	Payment	The final fare is based on the actual distance and time.
FR-13	Payment	The user pays by UPI, card, wallet or cash and gets a receipt.
FR-14	Feedback	The user rates the driver from 1 to 5 stars and can leave a comment.
FR-15	History	Users and drivers can look back at their past rides.
FR-16	Admin	The admin manages accounts, payments, refunds and bookings.

3.2 Technical Issues

- Live location needs a constant connection, so we use WebSockets instead of polling the server.
- Google Maps charges after a free limit, so routes and searches are cached.
- GPS can drift near tall buildings, so users can drag the pickup pin to the right spot.

3.3 Non-Functional Requirements

ID	Type	Requirement
NFR-01	Security	Passwords are hashed with bcrypt and all traffic uses HTTPS.
NFR-02	Privacy	Phone numbers are hidden, and calls go through the app.
NFR-03	Performance	A driver is assigned within 30 seconds on a normal day.
NFR-04	Performance	The cab's position on the map updates every 3 to 5 seconds.
NFR-05	Performance	Pages open within 3 seconds on 4G.
NFR-06	Reliability	99.9% uptime, daily backups, and nobody is charged twice for one ride.
NFR-07	Usability	Someone using the app for the first time can book a ride in under a minute.
NFR-08	Scalability	Handles 1,000 requests at once and grows by adding more servers.
NFR-09	Maintainability	Login, booking, rides and payments are separate modules.

3.4 Business Rules

Rule	Details
Search radius	3 km around the pickup first, then 5 km if nobody is free.
Driver reply time	15 seconds, then the request moves to the next driver.
Search timeout	After 2 minutes the user is told no cab is available.
Free cancellation	Within 2 minutes of booking, or if the driver is more than 10 minutes late.
Cancellation fee	Rs. 50 in other cases, added to the user's next ride.
Waiting charge	The first 3 minutes at pickup are free, then Rs. 1 per minute.
Login OTP	Works for 5 minutes, with at most 3 wrong tries.
Surge pricing	Up to 2x at peak times, and the user sees it before booking.
Driver rating	Drivers who fall below 3.5 stars on average get reviewed by the admin.

3.5 Fare Calculation

The fare is worked out when the ride ends:

$$\text{Fare} = \text{Base Fare} + (\text{Distance} \times \text{Rate per km}) + (\text{Ride Time} \times \text{Rate per min})$$

$$\text{Total} = (\text{Fare} \times \text{Surge}) + \text{Waiting Charge} + \text{Tolls} + \text{Taxes}$$

These are the sample rates we used for the project:

Vehicle	Seats	Base Fare	Per km	Per min
Auto	3	Rs. 30	Rs. 12	Rs. 1.00
Mini	4	Rs. 50	Rs. 14	Rs. 1.50
Sedan	4	Rs. 70	Rs. 17	Rs. 2.00
SUV	6	Rs. 100	Rs. 22	Rs. 2.50

So a 10 km Mini ride that takes 25 minutes costs $50 + (10 \times 14) + (25 \times 1.5) = \text{Rs. } 227.50$, before taxes and tolls.

4. External Interface Requirements

4.1 User Interfaces

Screen	Used By	What It Shows
Login / Sign up	User, Driver	Phone number, password and OTP
Home map	User	Pickup, drop, nearby cabs, and each vehicle type with its fare and ETA
Live ride	User	Driver details, live map and ride OTP
Payment	User	Final fare, payment options and receipt
Driver screens	Driver	Online switch, requests, directions, OTP entry, start and end ride
History	User, Driver	Past rides and receipts, plus earnings for drivers
Admin panel	Admin	Accounts, payments and live bookings

The layout is simple with big buttons, since drivers use it on the road, and actions like cancelling or ending a ride ask for confirmation first.

4.2 Hardware Interfaces

The app uses the phone's GPS, the camera for photos of driver documents, and SMS for OTPs. Nothing else is needed.

4.3 Communication Interfaces

- **HTTPS (REST):** normal requests between the app and the server, sent as JSON.
- **WebSockets:** live location and ride status.
- **SMS and email:** OTPs, receipts and account alerts.
- **Push notifications:** ride accepted, driver arriving and ride finished.

4.4 API Endpoints

Method	Endpoint	Purpose
POST	/api/auth/register	Sign up (an OTP is sent to the phone)
POST	/api/auth/login	Log in and get a token
GET	/api/fare-estimate	Fare and ETA for each vehicle type
POST	/api/bookings	Book a ride
PATCH	/api/bookings/{id}/cancel	Cancel a booking
POST	/api/rides/{id}/respond	Driver accepts or rejects
POST	/api/rides/{id}/start	Start the ride after the OTP check
POST	/api/rides/{id}/end	End the ride and get the final fare
POST	/api/payments	Pay for a ride
POST	/api/rides/{id}/rating	Rate the driver

5. Design

5.1 Use Case Diagram

Our three actors are the User, the Driver and the Administrator. The diagram shows what each of them can do in the app.

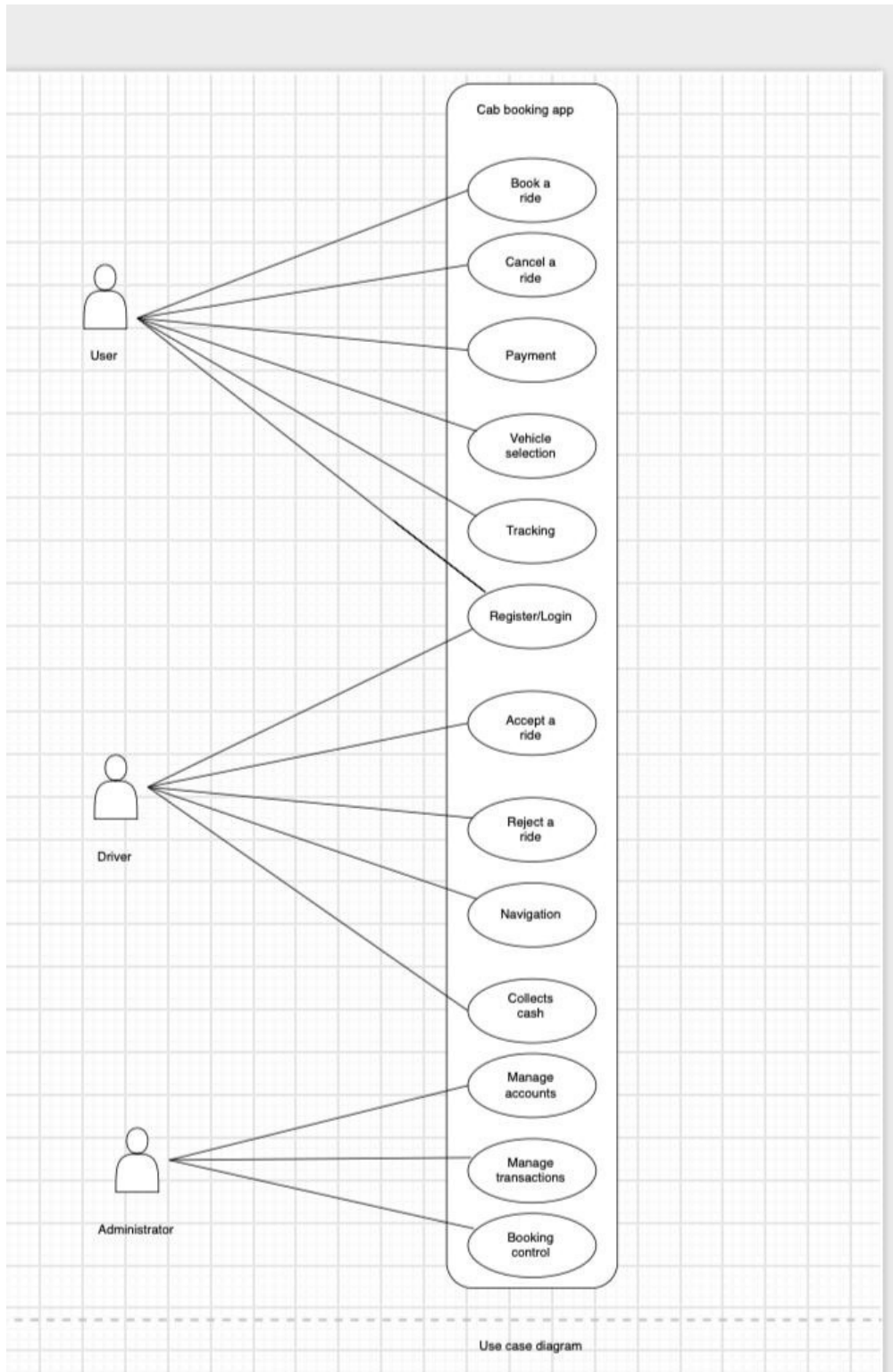


Fig. 5.1 Use Case Diagram

5.2 Use Case Descriptions

Here are the main use cases from the diagram in a bit more detail.

Use Case	Actor	Main Flow	Alternate Flow
UC-01: Register / Login	User, Driver	Enter name, phone and password → get an OTP → enter it → logged in	Wrong OTP: ask for a new one. Number already used: log in instead.
UC-02: Book a Ride	User	Enter pickup and drop → pick a vehicle → confirm → driver assigned → ride starts after the OTP	No driver accepts within 2 minutes: the user is told no cab is available.
UC-03: Cancel a Ride	User	Tap Cancel → pick a reason → rules checked → booking cancelled	Cancelled late: Rs. 50 is added to the next ride.
UC-04: Accept / Reject a Ride	Driver	See the request → tap Accept → user gets the driver's details → drive to pickup	Rejected or no reply: the request goes to the next driver.
UC-05: Make Payment	User, Driver	See the fare → choose a method → pay, or the driver marks cash → receipt	Payment fails: try again or pay in cash.
UC-06: Manage Accounts	Admin	Open users or drivers → check documents → approve or block	Unclear documents: the driver uploads them again.

5.3 Ride Status Flow

A booking moves through these statuses, and both the user and the driver can see the current one.

Status	Meaning
Requested	Booking confirmed, looking for a driver
Driver Assigned	A driver accepted and is on the way
Arrived	The driver is at the pickup point and waiting time starts
In Progress	OTP checked and the ride is going on
Completed	Ride ended and the fare is calculated
Paid	Payment received
Cancelled	Cancelled before the ride started
No Driver Found	Nobody accepted before the timeout

5.4 Activity Diagram: Booking a Ride

This is the user's side: checking if they're registered, entering the locations, waiting for a driver, then tracking the ride, paying and giving feedback.

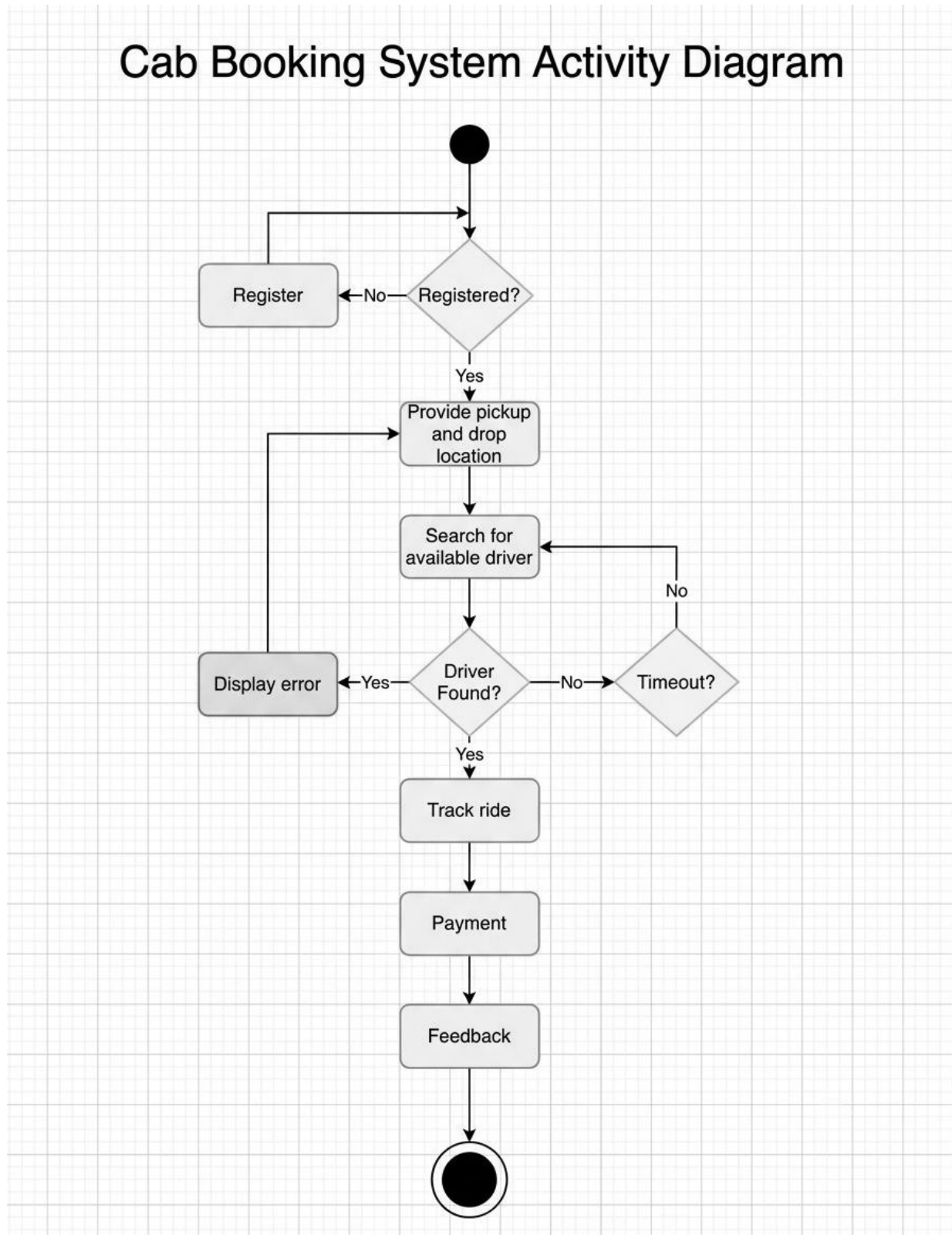


Fig. 5.2 Activity Diagram (User: booking a ride)

5.5 Activity Diagram: Handling a Ride

This one shows the driver's side, from going online and accepting a request to collecting the fare and going offline.

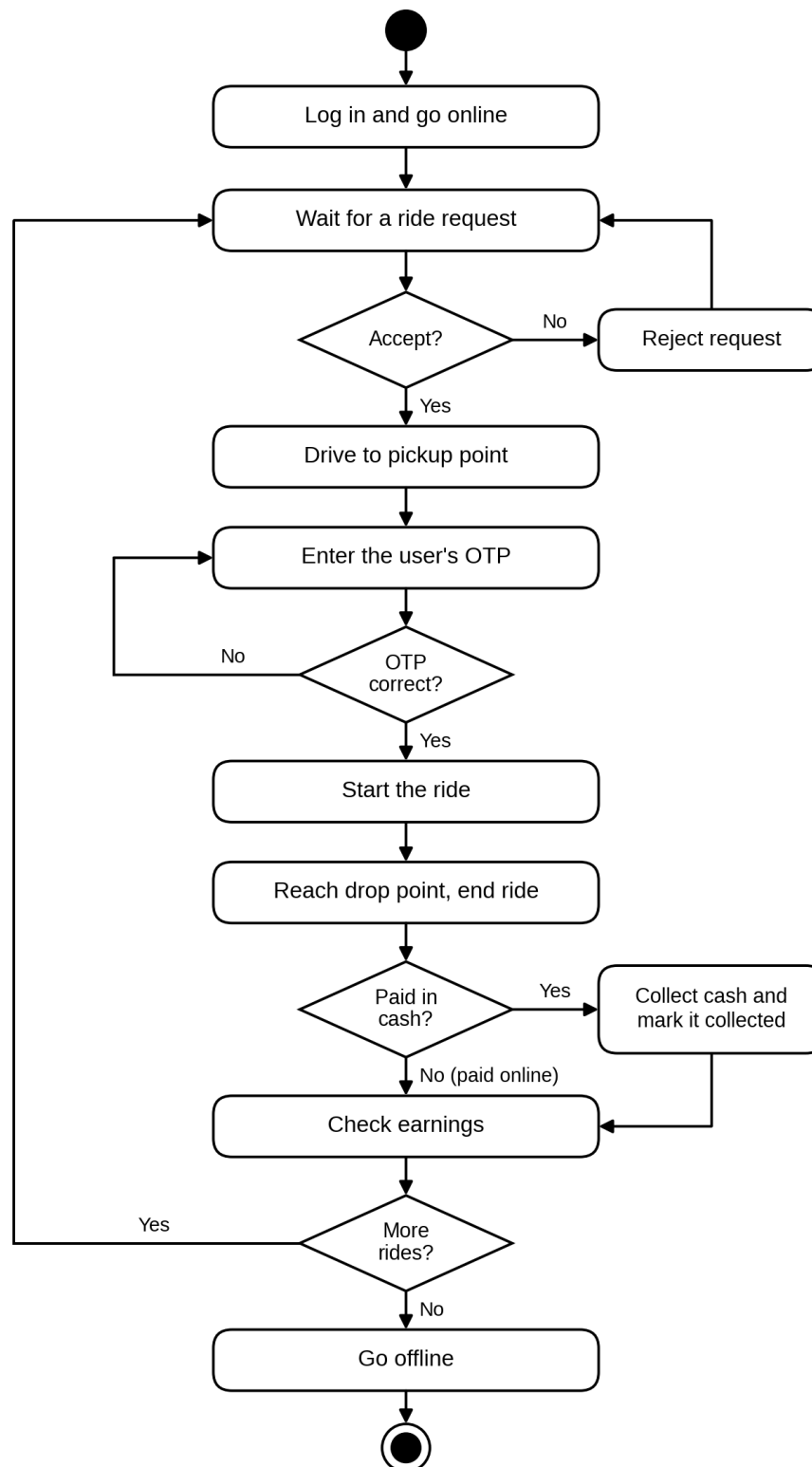


Fig. 5.3 Activity Diagram (Driver: handling a ride)

5.6 Data Flow Diagrams (DFD 0 and DFD 1)

DFD 0 treats the whole system as one process that talks to the User and the Driver. DFD 1 splits it into five processes (registration, booking, ride, driver and payment management) and shows where each one stores its data.

DFD 0 — Context Level

DFD 0-Context Level

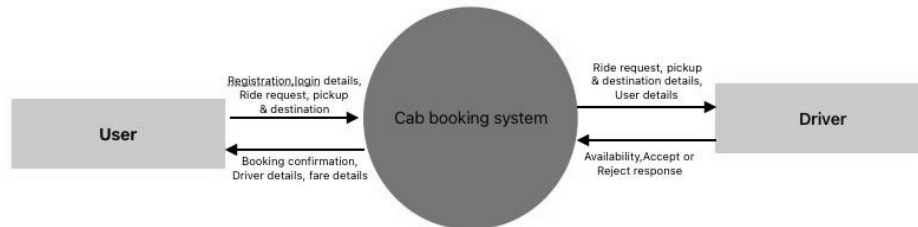


Fig. 5.4 DFD Level 0 (Context Diagram)

DFD 1 — Level 1

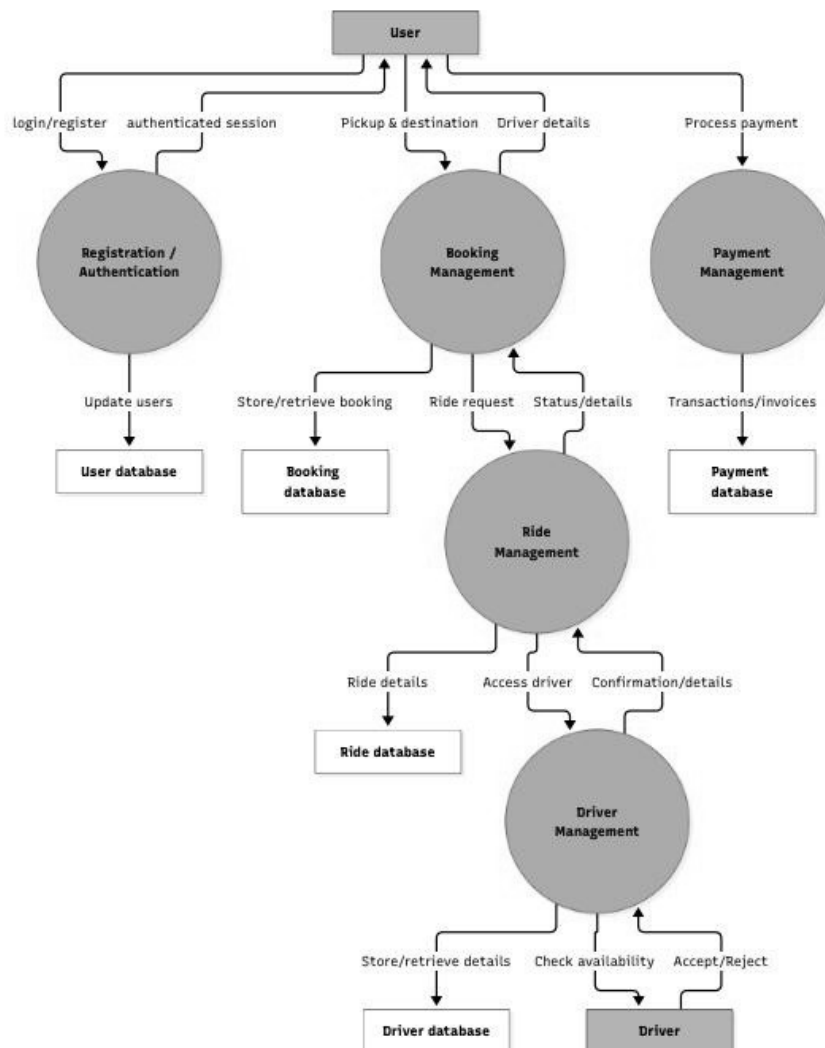


Fig. 5.5 DFD Level 1

5.7 DFD Level 2: Ride Management

DFD 2 zooms into Ride Management, the busiest process in DFD 1. We numbered it as process 4, so its parts are 4.1 to 4.5. It shows how a ride request turns into an assigned driver, then a started ride and finally a completed ride with a fare. Its inputs and outputs match the ones Ride Management has in DFD 1.

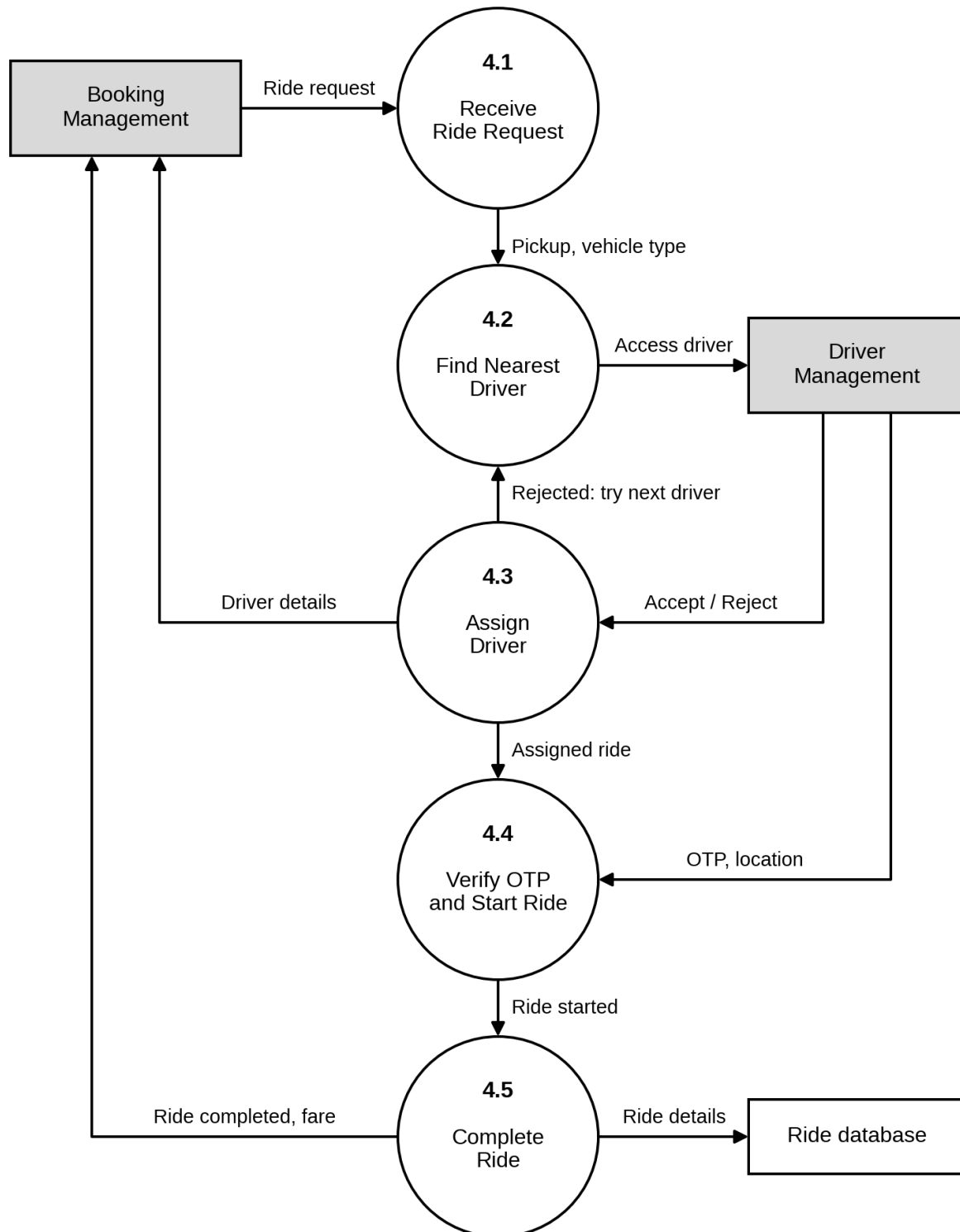


Fig. 5.6 DFD Level 2 (Ride Management)

6. Database Design

6.1 Tables

We need seven tables. Five of them match the data stores in DFD 1, and Vehicles and Ratings are extra. IDs are auto-generated, money is stored as NUMERIC(8,2) and times as TIMESTAMP.

Table	Main Fields	Stores
Users	user_id (PK), name, phone, email, password_hash, role	Riders and admins
Drivers	driver_id (PK), name, phone, licence_no, status, is_online, location, avg_rating	Driver profiles and live status
Vehicles	vehicle_id (PK), driver_id (FK), vehicle_no, type, model	Each driver's car or auto
Bookings	booking_id (PK), user_id (FK), pickup, drop, vehicle_type, estimated_fare, status	Ride requests
Rides	ride_id (PK), booking_id (FK), driver_id (FK), ride_otp, start_time, end_time, distance_km, final_fare, status	Actual trips
Payments	payment_id (PK), ride_id (FK), amount, method, status, transaction_ref	Payments and receipts
Ratings	rating_id (PK), ride_id (FK), stars, comment	Feedback on drivers

6.2 Relationships

- A user can make many bookings (1 : N).
- A booking becomes at most one ride (1 : 1).
- A driver has one vehicle (1 : 1) and does many rides (1 : N).
- A ride has one payment and at most one rating (1 : 1).

7. Testing

7.1 Test Cases

ID	Test	What We Do	What Should Happen
TC-01	Sign up	Enter correct details and OTP	Account created and logged in
TC-02	Wrong OTP	Enter a wrong OTP 3 times	Blocked until a new OTP is sent
TC-03	Fare estimate	Pick two places 10 km apart	Fare and ETA for every vehicle
TC-04	Same pickup and drop	Enter the same place twice	Booking not allowed
TC-05	Book a ride	Book with drivers nearby	Nearest driver assigned
TC-06	No drivers	Book when all drivers are offline	"No cab available" after 2 minutes
TC-07	Driver rejects	Driver taps Reject	Request goes to the next driver
TC-08	Wrong ride OTP	Driver types a wrong OTP	Ride doesn't start
TC-09	Late cancel	Cancel after the driver arrives	Rs. 50 fee added
TC-10	Payment fails	Gateway returns an error	Retry or pay cash, no double charge
TC-11	Rate driver	Give 4 stars	Average rating updated
TC-12	Blocked driver	A blocked driver logs in	Access denied

7.2 Requirement Traceability

This links each requirement to its use case and the tests that check it, so nothing is left untested.

Area	Requirements	Use Case	Test Cases
Sign up and login	FR-01, FR-03	UC-01	TC-01, TC-02
Fare estimate	FR-04, FR-05	UC-02	TC-03, TC-04
Booking and matching	FR-06, FR-08	UC-02	TC-05, TC-06
Accept / reject	FR-09	UC-04	TC-07
Ride OTP	FR-10	UC-02	TC-08
Cancellation	FR-07	UC-03	TC-09
Payment	FR-12, FR-13	UC-05	TC-10
Rating	FR-14	UC-05	TC-11
Admin approval	FR-02, FR-16	UC-06	TC-12

8. Risk Analysis

Risk	Effect	How We Handle It
Payment gateway down	Can't pay online	Cash still works, and unpaid rides can be paid later from history.
Rush-hour load	Slow or crashing server	A load balancer and auto-scaling on the cloud.
Driver doesn't show up	User is left waiting	Free cancellation after 10 minutes. Drivers who do this often get fewer requests.
Fake bookings	Drivers waste time and fuel	Cancellation fee, and the admin blocks repeat offenders.
Internet drops mid-ride	Tracking stops	The driver's app keeps the GPS points and sends them later, so the fare is still right.

9. Future Scope

- Ride sharing, so people going the same way can split the fare.
- Separate Android and iOS apps.
- Electric cabs, with nearby charging points shown to drivers.
- Safety features like an SOS button and live trip sharing.
- Using past ride data to predict busy areas and send drivers there early.

10. Conclusion

This SRS sums up what our cab booking app needs to do and how it should behave. The requirements, diagrams, database design and test cases together give us a clear plan for building the app, and a checklist to test the finished version against.

11. References

- IEEE Std 830-1998, IEEE Recommended Practice for Software Requirements Specifications
- Roger S. Pressman, Software Engineering: A Practitioner's Approach
- PostgreSQL Documentation, www.postgresql.org/docs
- developers.google.com/maps
- www.geeksforgeeks.org